Introduction to Computer Architecture and Systems

Chapter 5
Control Flows, Hazards

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- Writing snippets: why, when, how
- 2’s Complement helper program in LC-3
- Introduction to Control Flow
- 2 ways of writing loops: counter and sentinel

Previously of CSCI 260…

- Systematic Decomposition
- New Program: Counting Occurrence of a Character

Cover-ups…

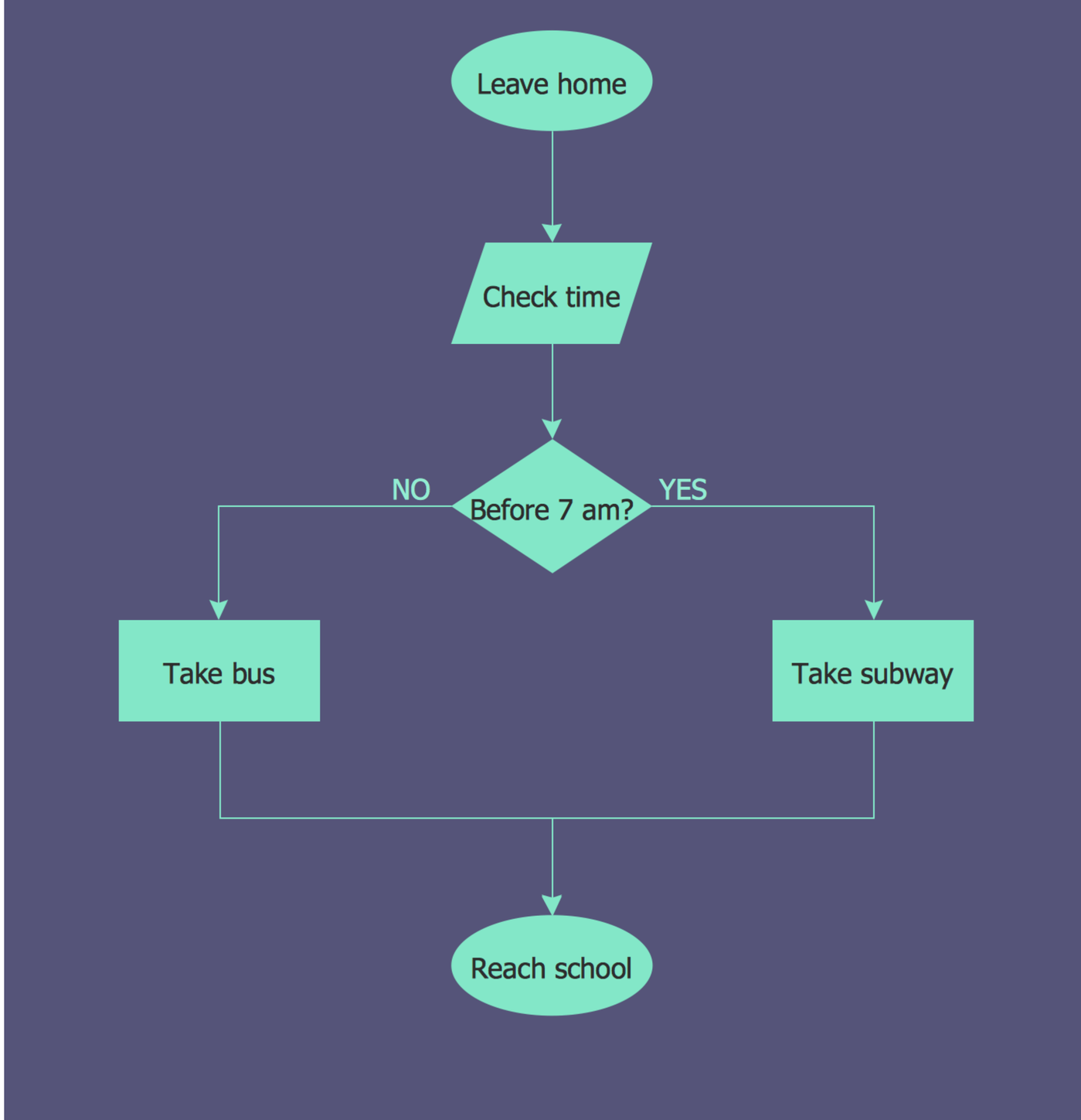
Cover-Up for TRAP

New preloaded programs…

- x23 = Input a character from keyboard
- x21 = Output a character to the monitor
- x25 = Halt the program

Flowchart

- Circles: Start / End States
- Parallelograms: Input / Output
- Diamonds: Condition
- Rectangles: Actions



Assignment 1

Assignment 1

Fibonacci Sequence

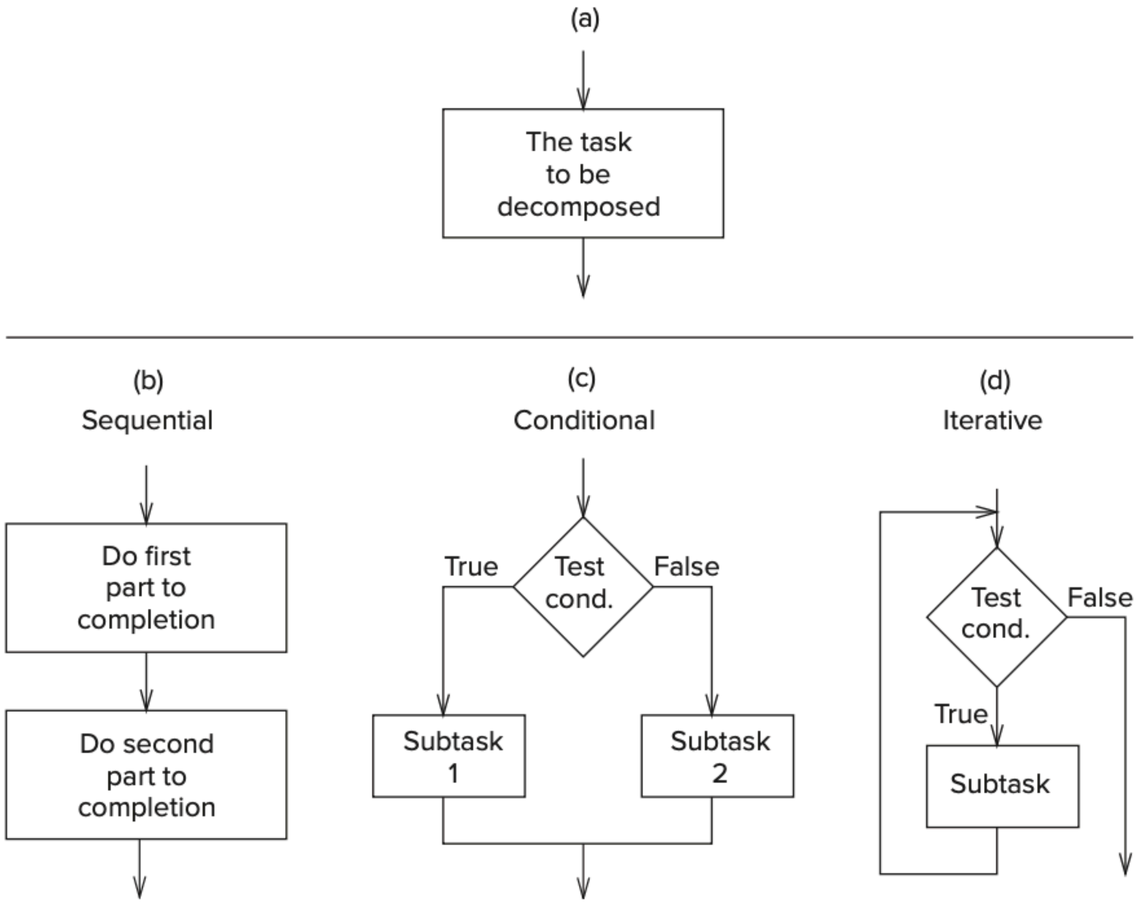
- Write in LC-3 Assembly to **calculate the 20th number of fibonacci sequence; calculate nth number of the sequence if you can.**
- Use 5-step approach to write code, and run it on LC3Tools.
- Submit your code in coding form, and a screenshot of LC3Tools.
- Assignment is releasing tonight, and due 23:50 this Sunday(10/05).
- Seek help with your classmates, and visit my office hours, if in doubt.

Systematic Decomposition

Another Missing Piece…

Control flow when breaking down the problem…

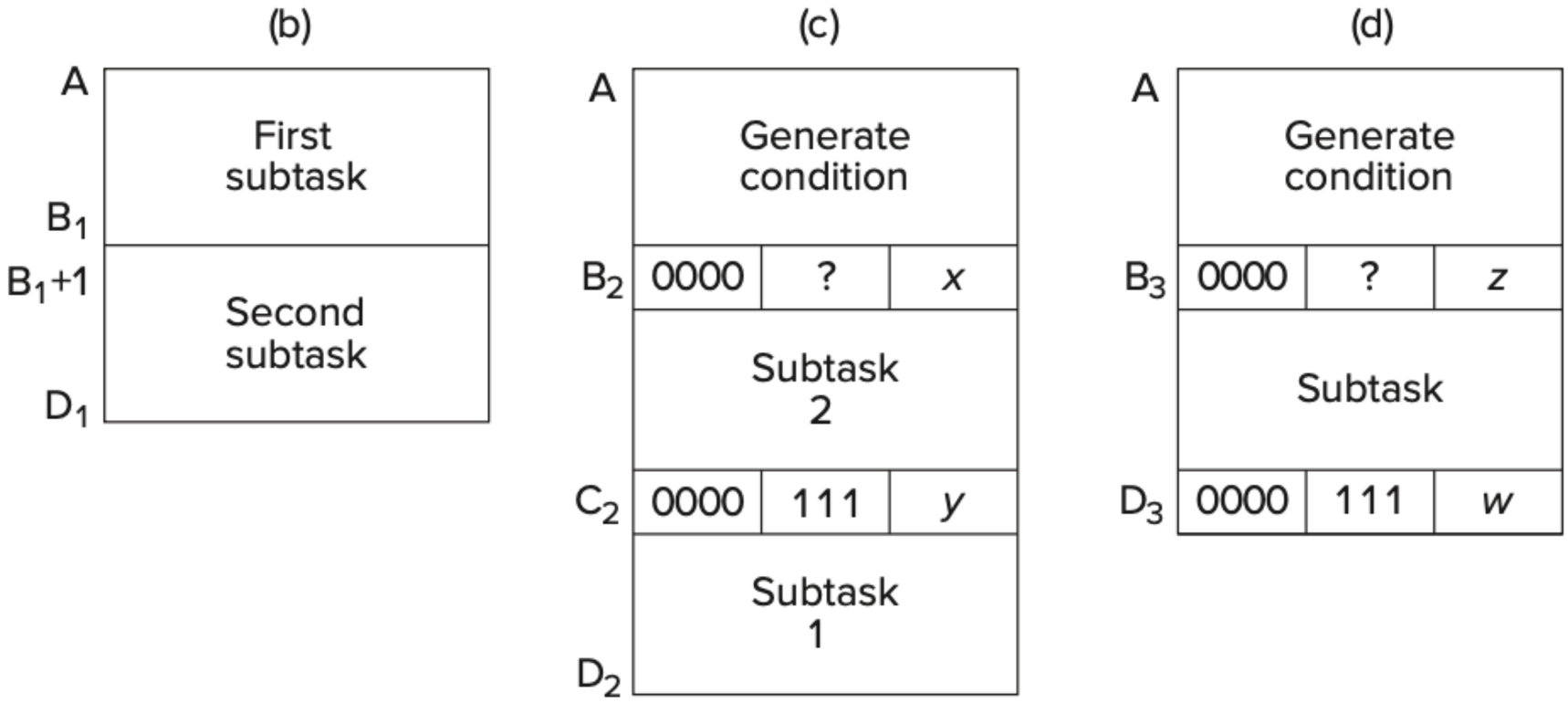
- 3 types of control flow…



Another Missing Piece…

Control flow when breaking down the problem…

- 3 types of code organization…



Example: Counting Occurrence of a Character

Counting Occurrence of a Character

Before starting…

- New Sentinel: b00000100 = End of Transmission(EOT) from ASCII.
- Counting occurrence of a character = find matches in ASCII values until EOT.
- Break down the problem:
  - How to break down the problem so subproblems reaches to ISA level?
  - How to organize the control flow?
  - How to initialize values?
  - How to read the word and display the result?

Don’t forget your survival kit…

To lookup ASCII and ISAs

E.2 Standard ASCII codes

Table E.2 The Standard ASCII Table

ASCII			ASCII			ASCII			ASCII		
Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex
nul	0	00	sp	32	20	@	64	40	'	96	60
soh	1	01	!	33	21	A	65	41	a	97	61
stx	2	02	"	34	22	B	66	42	b	98	62
etx	3	03	#	35	23	C	67	43	c	99	63
eot	4	04	$	36	24	D	68	44	d	100	64
enq	5	05	%	37	25	E	69	45	e	101	65
ack	6	06	&	38	26	F	70	46	f	102	66
bel	7	07	'	39	27	G	71	47	g	103	67
bs	8	08	(	40	28	H	72	48	h	104	68
ht	9	09	)	41	29	I	73	49	i	105	69
lf	10	0A	*	42	2A	J	74	4A	j	106	6A
vt	11	0B	+	43	2B	K	75	4B	k	107	6B
ff	12	0C	'	44	2C	L	76	4C	l	108	6C
cr	13	0D	-	45	2D	M	77	4D	m	109	6D
so	14	0E	.	46	2E	N	78	4E	n	110	6E
si	15	0F	/	47	2F	O	79	4F	o	111	6F
dle	16	10	0	48	30	P	80	50	p	112	70
dc1	17	11	1	49	31	Q	81	51	q	113	71
dc2	18	12	2	50	32	R	82	52	r	114	72
dc3	19	13	3	51	33	S	83	53	s	115	73
dc4	20	14	4	52	34	T	84	54	t	116	74
nak	21	15	5	53	35	U	85	55	u	117	75
syn	22	16	6	54	36	V	86	56	v	118	76
etb	23	17	7	55	37	W	87	57	w	119	77
can	24	18	8	56	38	X	88	58	x	120	78
em	25	19	9	57	39	Y	89	59	y	121	79
sub	26	1A	:	58	3A	Z	90	5A	z	122	7A
esc	27	1B	;	59	3B	[	91	5B	{	123	7B
fs	28	1C	<	60	3C	\	92	5C	|	124	7C
gs	29	1D	=	61	3D	]	93	5D	}	125	7D
rs	30	1E	>	62	3E	^	94	5E	~	126	7E
us	31	1F	?	63	3F	_	95	5F	del	127	7F

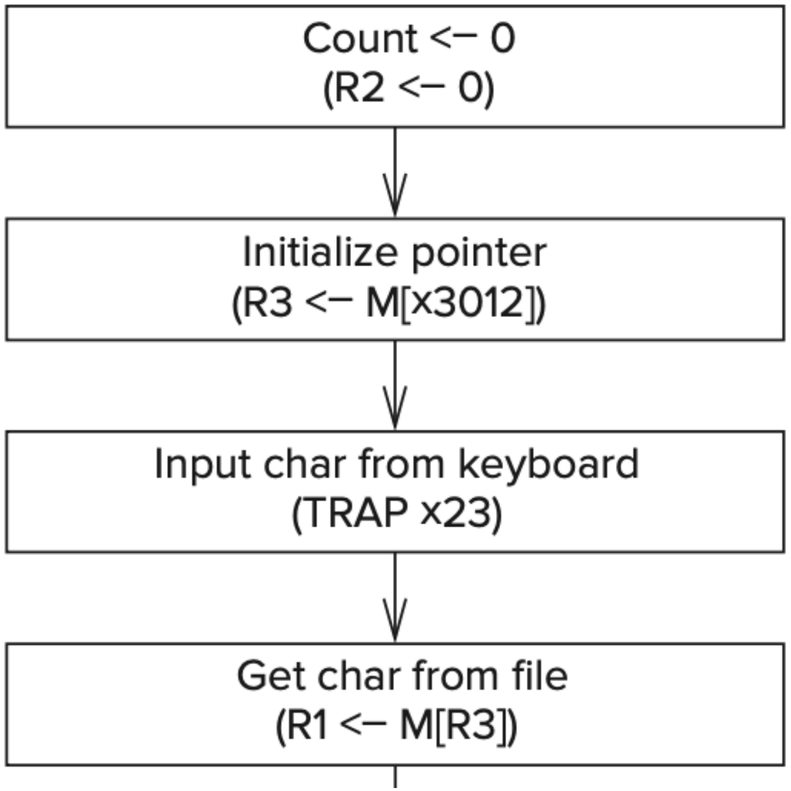
Counting Occurrence of a Character

Determining algorithm / solution

- Breaking down the problem
- Assigning resources
- Draw flowchart

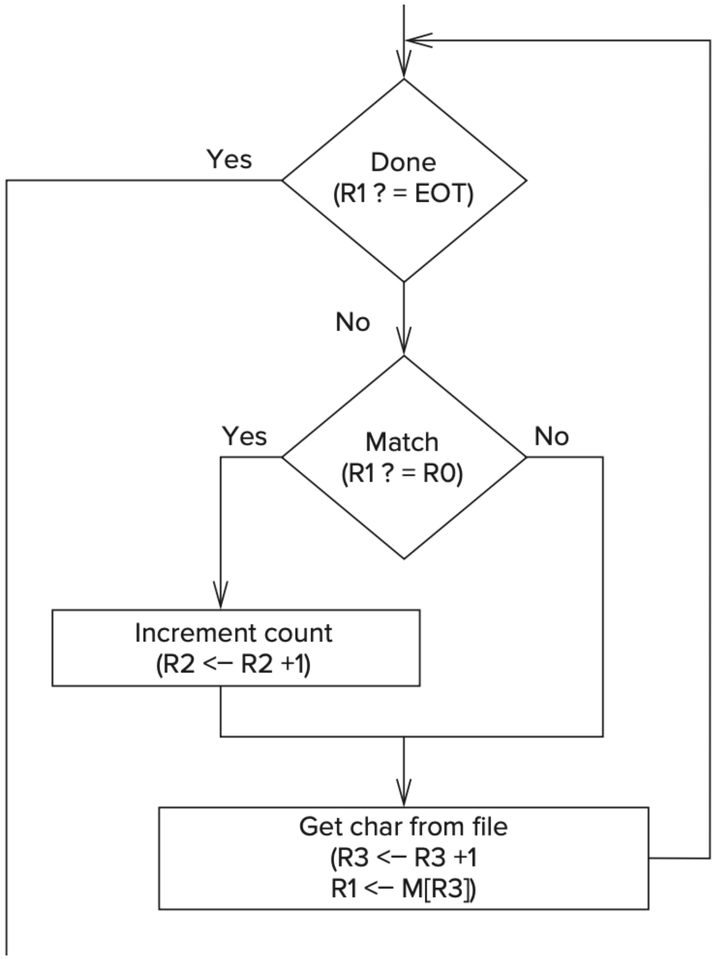
Counting Occurrence of a Character

Part 1: Get Input; Initialize Counter



Counting Occurrence of a Character

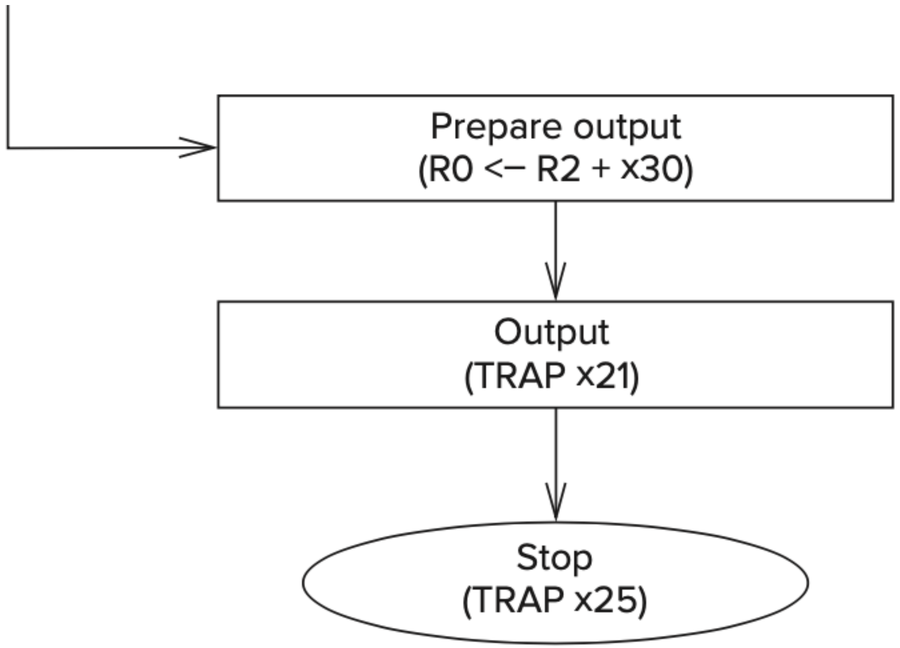
Part 2: Process inputs; counting matches



Counting Occurrence of a Character

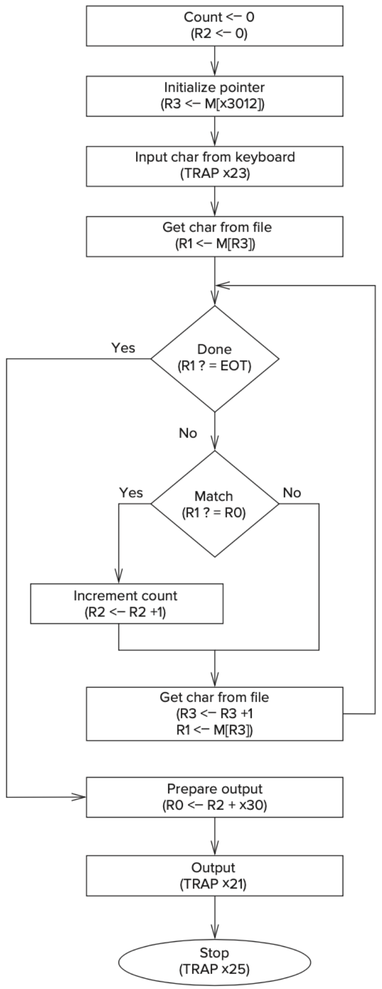
Part 3: Prepare Outputs, Finishing Up

If we reached EOT…



Counting Occurrence of a Character

Big Picture, then assign corresponding resources…



How many registers do we need?

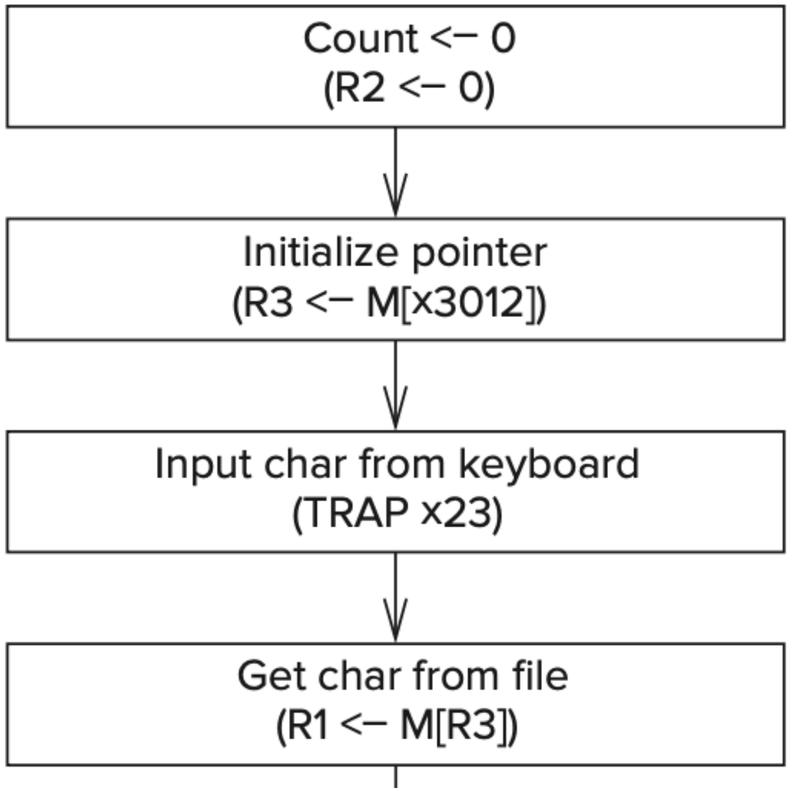
How to call the loops?

How to make branches?

Answer

Part 1: Preparing

Counter, Pointer, and the Character Looking For

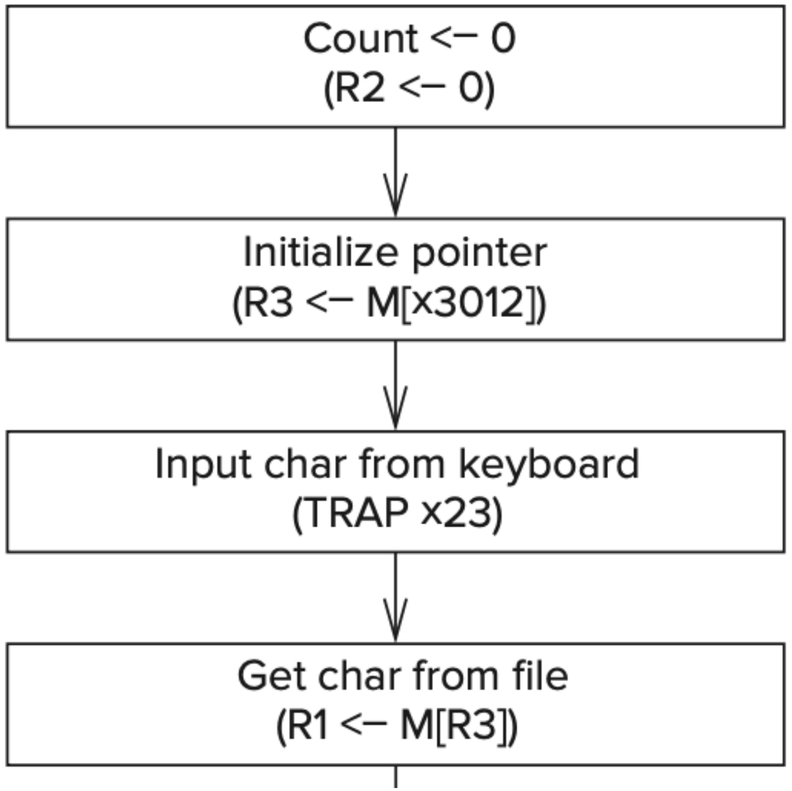


- R2 <- 0
- R3 <- M[3012]
- TRAP x23
- R1 <- M[R3]

- Counter: R2 = 0
- R3 set to begin
- Get input
- Read via Pointer

Part 1: Preparing

Counter, Pointer, and the Character Looking For

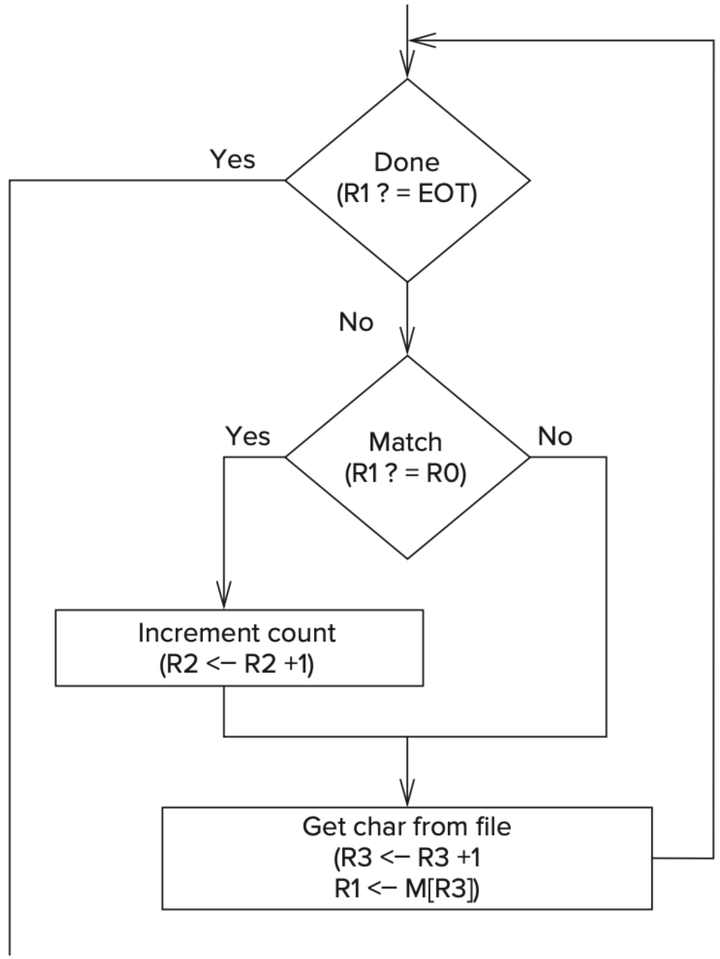


- R2 <- 0
- R3 <- M[3012]
- TRAP x23
- R1 <- M[R3]

- AND R2 R2 0
- LD R3 16
- TRAP x23
- LDR R1 R3 0

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match



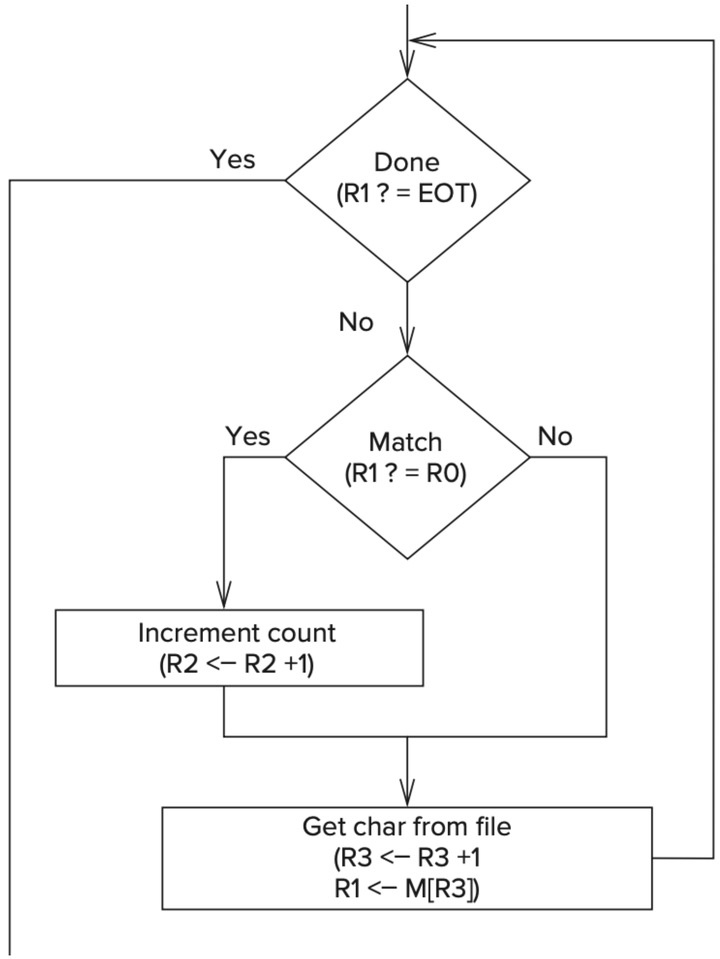
- R4 <- R1 - 4 (EOT)
- BRz x300E
- R1 <- NOT R1
- R1 <- R1 + 1
- R1 <- R1 + R0
- BRnp x300B

- Check EOT
- If so, Part 3
- R1 on negative…

Check if R1 = R0 (Match)
If not, move R3 on

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match

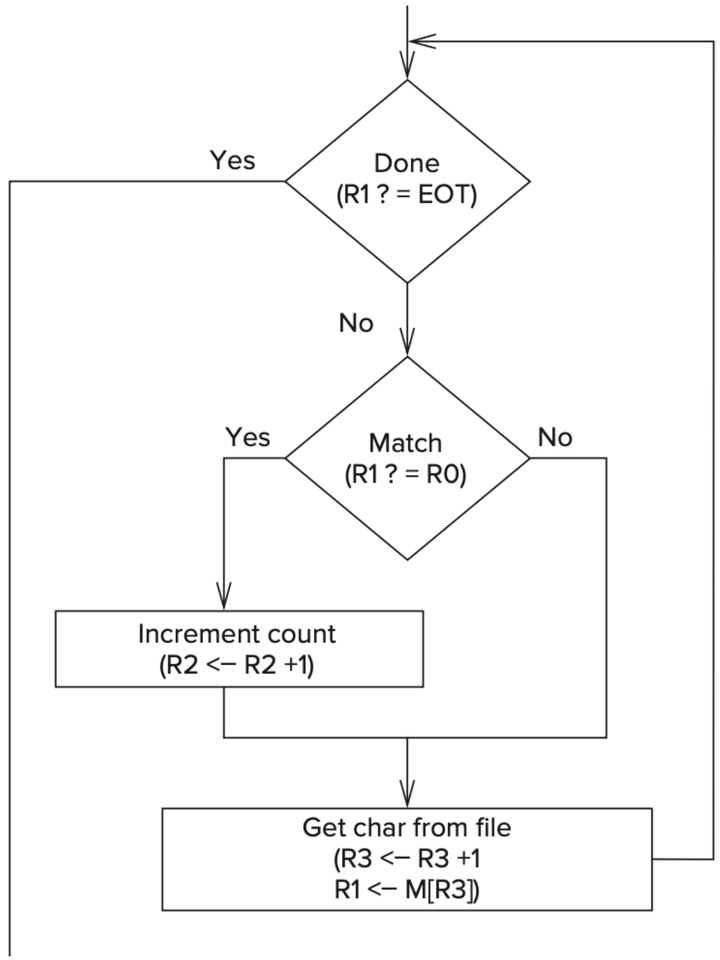


- R4 <- R1 - 4 (EOT)
- BRz x300E
- R1 <- NOT R1
- R1 <- R1 + 1
- R1 <- R1 + R0
- BRnp x300B

- ADD R4 R1 -4
- BRz 8
- NOT R1 R1 11111
- ADD R1 R1 1
- ADD R1 R1 R0
- BRnp 1

Part 2: Process inputs; counting matches

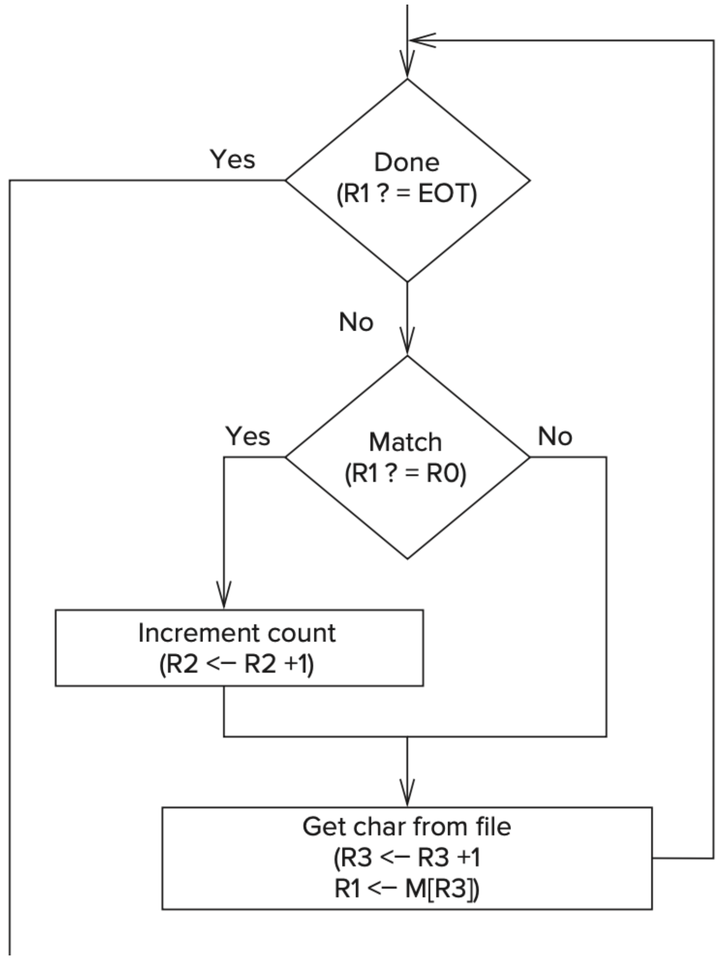
Outside Loop: If Done; Inside Loop: If got match



- R2 <- R2 + 1 If match, add R2…
- R3 <- R3 + 1 And move R3.
- R1 <- M[R3] Load next character
- BRnzp M[x3004] Send back to outer loop

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match

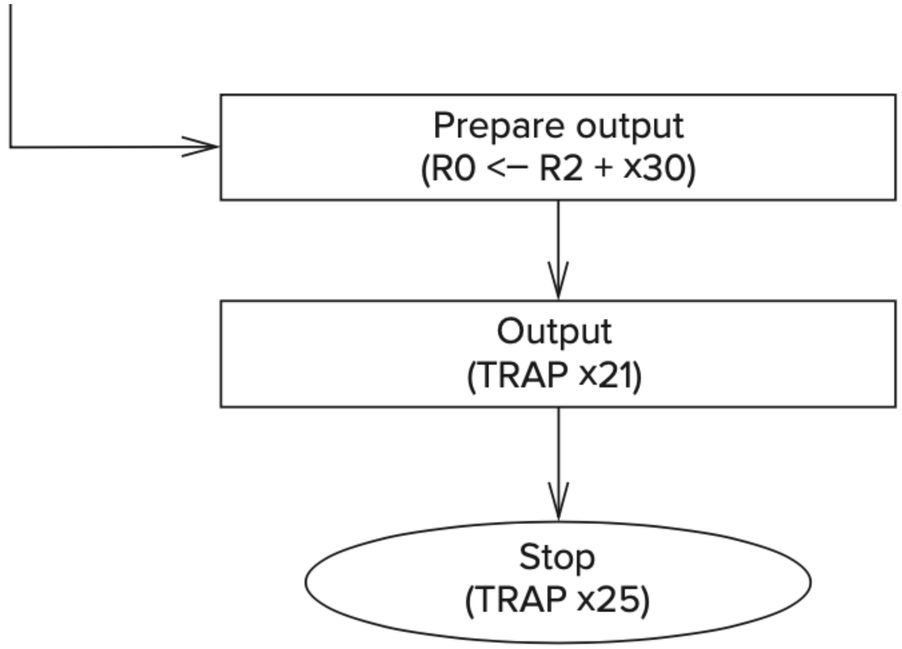


- R2 <- R2 + 1 — ADD R2 R2 1
- R3 <- R3 + 1 — ADD R3 R3 1
- R1 <- M[R3] — LD R1 R3
- BRnzp M[x3004] — BRnzp -10

Counting Occurrence of a Character

Part 3: Prepare Outputs, Finishing Up

If we reached EOT…

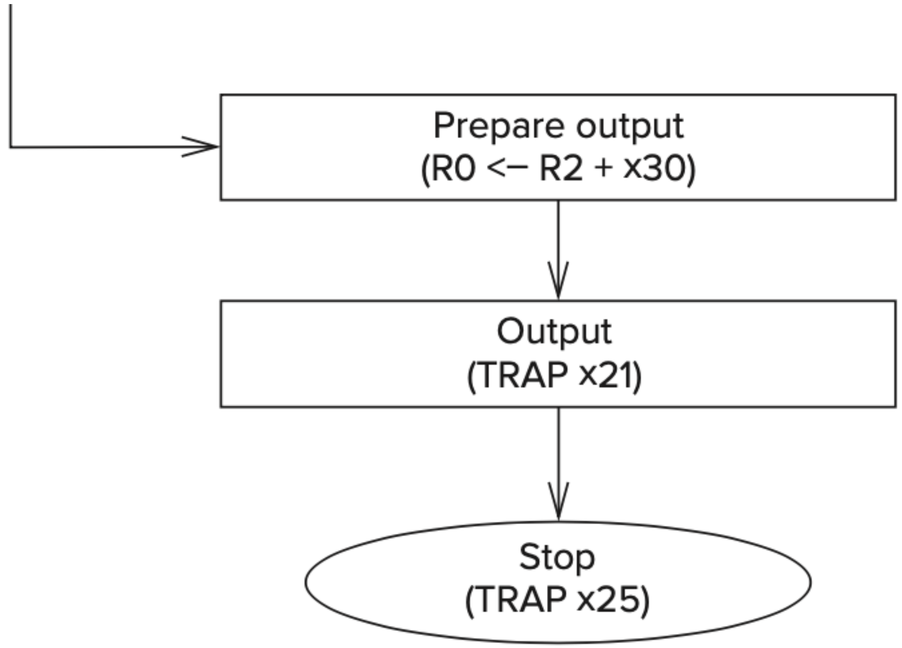


- R0 <- M[x3013] ASCII ‘0’ position
- R0 <- R0 + R2 Counter in ASCII
- TRAP x21 TRAP x21
- TRAP x25 TRAP x25

Counting Occurrence of a Character

Part 3: Prepare Outputs, Finishing Up

If we reached EOT…



- R0 <- M[x3013] — LD R0 4
- R0 <- R0 + R2 — ADD R0 R2
- TRAP x21 — TRAP x21
- TRAP x25 — TRAP x25

Complete Program

	Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
	x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	`R2 <- 0`
	x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	`R3 <- M[x3012]`
	x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	`TRAP x23`
	x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
ASCII x4 ('EOT')	x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	`R4 <- R1-4`
	x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	`BRz x300E`
	x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	`R1 <- NOT R1`
	x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	`R1 <- R1 + 1`
	x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	`R1 <- R1 + R0`
	x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	`BRnp x300B`
	x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	`R2 <- R2 + 1`
	x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	`R3 <- R3 + 1`
	x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
	x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	`BRnzp x3004`
	x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	`R0 <- M[x3013]`
	x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	`R0 <- R0 + R2`
	x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	`TRAP x21`
	x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	`TRAP x25`
	x3012	Starting address of file																
ASCII x30 ('0')	x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	`ASCII TEMPLATE`

Pipeline Hazard

Pipeline Hazard: Things prevent instructions aligning with instruction cycle in pipelined fashion

Clock Cycle and Instruction Cycle

What is a Cycle?

Assumptions and Loopholes

- Actually, steps in different instructions in their instruction cycles, are not taking the same time.
- For simplicity, when we talk about instructions in instruction cycle, we take them as 1 cycle for each stage.
- Several observation holds:
  1. At ISA level, each stage takes time small enough to ignore the difference.
  2. Memory Hierarchy gives differences in time (eg: visiting register takes 1 clock cycle, visiting memory takes 10~100).
  3. Instruction cycle is not equal to clock cycle: clock cycle is the shortest, undivided time within the processor.

3 Types of Pipeline Hazard…

Structural Hazard
Data Hazard
Control Hazard

Data Hazard

Data Hazard

Concurrency of Data

- A data hazard occurs when the pipeline execution must be stalled because one step must wait for another one to complete.
- This comes up when a planned instruction can not execute in the planned clock cycle because the data needed is not yet available.



Data Hazard

4 Types of Data Dependencies

- Read after Write (RAW)
  It is also known as True dependency or Flow dependency. It occurs when the value produced by an instruction is required by a subsequent instruction.

```
ADD R1, --, --;
ADD --, R1, --;
```

- Write after Read (WAR)
  It is also known as anti dependency. These hazards occur when the output register of an instruction is used right after read by a previous instruction.

```
ADD --, R1, --;
ADD R1, --, --;
```

Data Hazard

4 Types of Data Dependencies

- Write after Write (WAW)
  It is also known as output dependency. These hazards occur when the output register of an instruction is used for write after written by previous instruction.

```
ADD R1, --, --;
ADD R1, --, --;
```

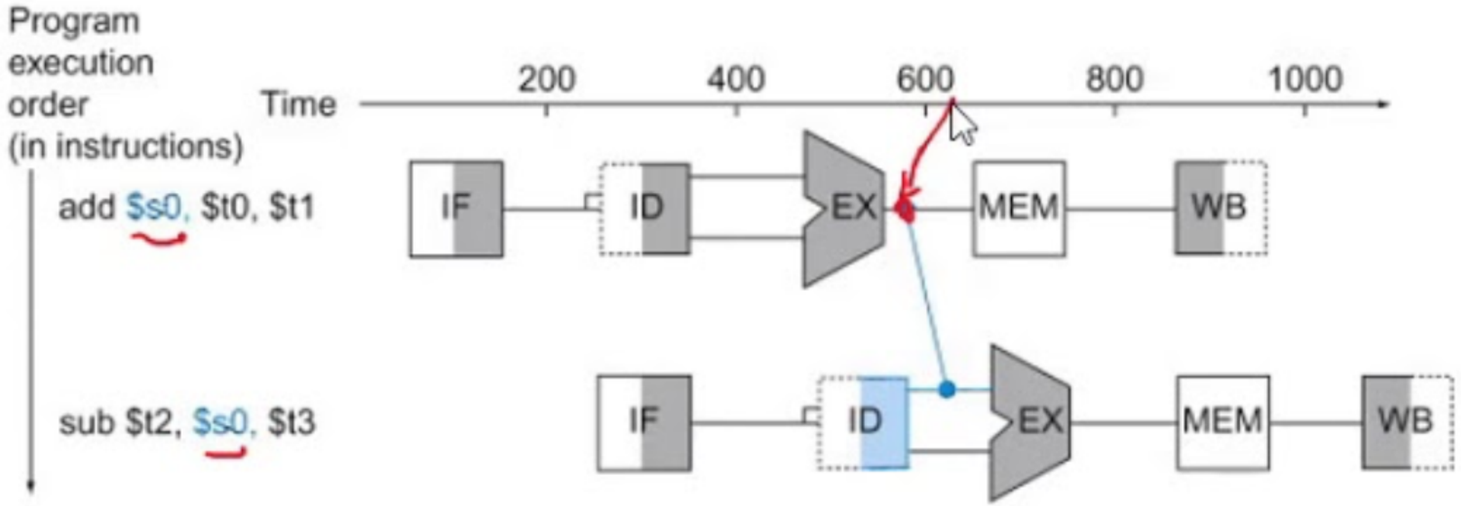
- Read after Read (RAR) :
  It occurs when the instruction both read from the same register.
  Since reading a register value does not change the register value, these Read after Read (RAR) hazards don't cause a problem for the processor.

```
ADD --, R1, --;
ADD --, R1, --;
```

Handling Data Hazards

Change the hardware, change the code, or…

- Forwarding :
  It adds special circuitry to the pipeline. This method works because it takes less time for the required values to travel through a wire than it does for a pipeline segment to compute its result.



Handling Data Hazards

Change the hardware, change the code, or…

- Code reordering :
  We need a special type of software to reorder code. We call this type of software a hardware-dependent compiler.

```
ADD R0 R1 R2
ADD R2 R2 R1
LD  R5 M[x3100]
ADD R2 R0 R5
```

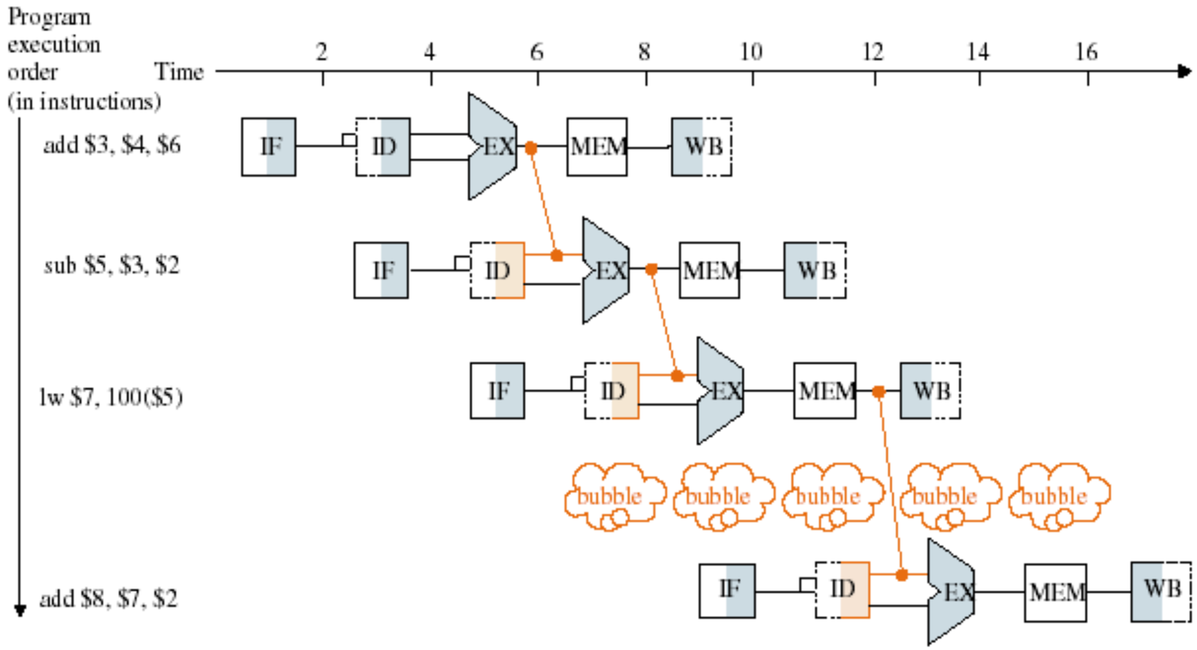


```
ADD R0 R1 R2
LD  R5 M[x3100]
ADD R2 R2 R1
ADD R2 R0 R5
```

Handling Data Hazards

Change the hardware, change the code, or…

- Stall Insertion :
  it inserts one or more stall (no-op instructions) into the pipeline, which delays the execution of the current instruction until the required operand is written to the register file, but this method decreases pipeline efficiency and throughput.



Control Hazard

Control Hazard

Time and Effort Lost with Wrong Prediction

- An instruction that is control dependent on a branch cannot be moved in front of the branch, so that the branch no longer controls it and an instruction that is not control dependent on a branch cannot be moved after the branch so that the branch controls it. This will give rise to control hazards.

- Wrong branch predictions will cause two problems if not handled well: exception behavior and handling and preservation of data flow.

```
ADD R2, R3, R4;
LD  R1, R2
BRz R2, #3;
```

Control Hazard

Time and Effort Lost with Wrong Prediction

- Stall until the branch outcome is known or perform the fetch again
- Predict the behavior of branches… better, with…
  - Static prediction by the compiler
  - Dynamic prediction by the hardware:
    - Predict not taken approach
      Assume that the branch is not taken, i.e. the condition will not evaluate to be true
    - Predict taken approach
      Assume that the branch is taken, i.e. the condition will evaluate to be true
    - Delayed branching
      A more effective solution

Data Hazard

4 Types of Data Dependencies

- Read after Write (RAW)
  It is also known as True dependency or Flow dependency. It occurs when the value produced by an instruction is required by a subsequent instruction.

```
ADD R1, --, --;
ADD --, R1, --;
```

- Write after Read (WAR)
  It is also known as anti dependency. These hazards occur when the output register of an instruction is used right after read by a previous instruction.

```
ADD --, R1, --;
ADD R1, --, --;
```

Data Hazard

4 Types of Data Dependencies

- Write after Write (WAW)
  It is also known as output dependency. These hazards occur when the output register of an instruction is used for write after written by previous instruction.

```
ADD R1, --, --;
ADD R1, --, --;
```

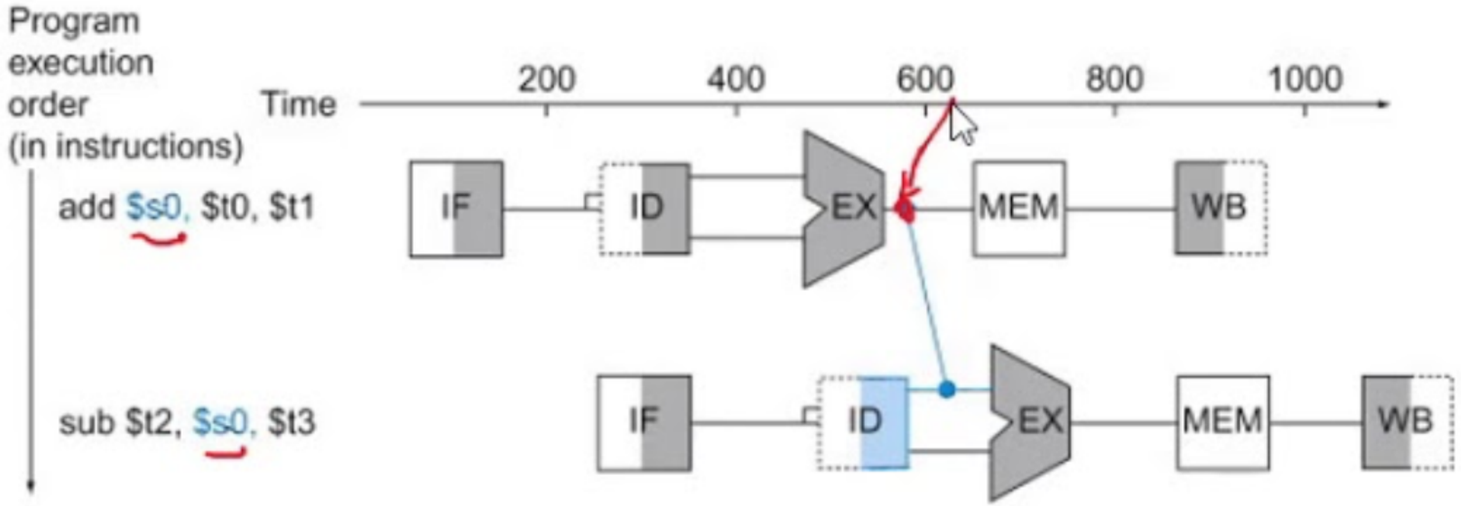
- Read after Read (RAR) :
  It occurs when the instruction both read from the same register.
  Since reading a register value does not change the register value, these Read after Read (RAR) hazards don't cause a problem for the processor.

```
ADD --, R1, --;
ADD --, R1, --;
```

Handling Data Hazards

Change the hardware, change the code, or…

- Forwarding :
  It adds special circuitry to the pipeline. This method works because it takes less time for the required values to travel through a wire than it does for a pipeline segment to compute its result.



Handling Data Hazards

Change the hardware, change the code, or…

- Code reordering :
  We need a special type of software to reorder code. We call this type of software a hardware-dependent compiler.

```
ADD R0 R1 R2
ADD R2 R2 R1
LD    R5 M[x3100]
ADD R2 R0 R5
```

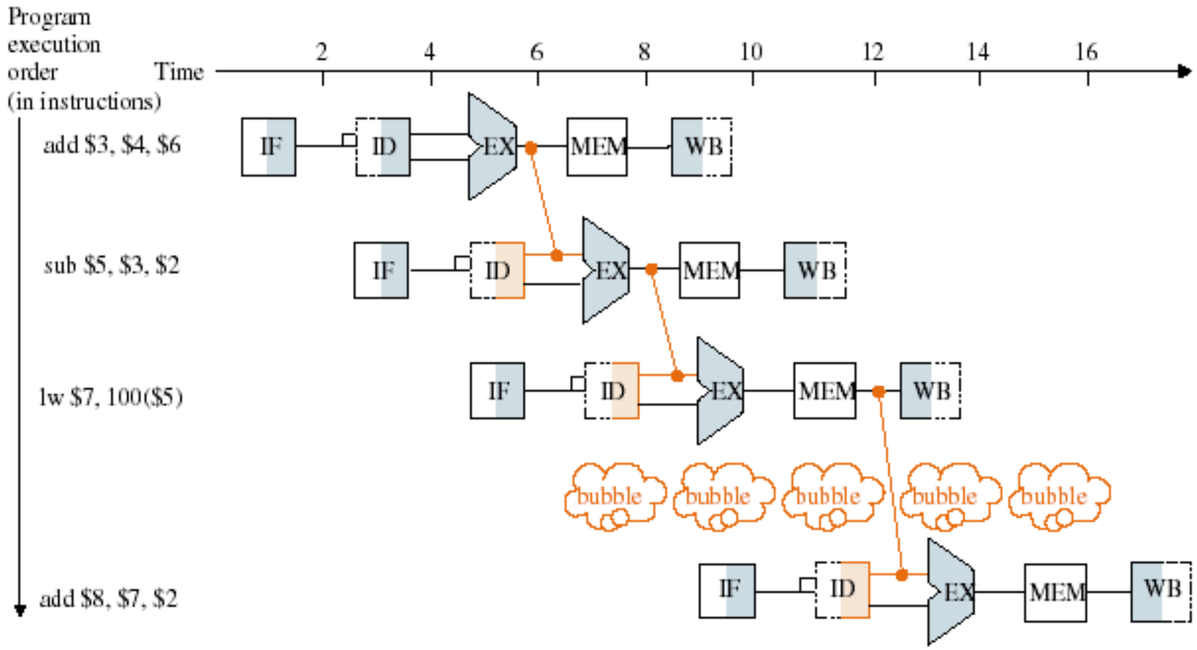


```
ADD R0 R1 R2
LD    R5 M[x3100]
ADD R2 R2 R1
ADD R2 R0 R5
```

Handling Data Hazards

Change the hardware, change the code, or…

- Stall Insertion :
  it inserts one or more stall (no-op instructions) into the pipeline, which delays the execution of the current instruction until the required operand is written to the register file, but this method decreases pipeline efficiency and throughput.



Control Hazard

Time and Effort Lost with Wrong Prediction

- An instruction that is control dependent on a branch cannot be moved in front of the branch, so that the branch no longer controls it and an instruction that is not control dependent on a branch cannot be moved after the branch so that the branch controls it. This will give rise to control hazards.
- Wrong branch predictions will cause two problems if not handled well: exception behavior and handling and preservation of data flow.

```
if(a > 0) {
    action(1);
  } else {
    action(2);
}
```

```
ADD R2, R3, R4;
LD  R1, R2
BRz R2, #3;
```

Why Control Hazard a Real Problem?

- Assumption: A PC change by condition is done in the n-th pipeline stage.
- Consequence: n−1 instructions after the jump are already in the pipeline.
- Therefore, n−1 instructions must be cancelled (pipeline flush).

- Consider in the real world, every 4-6 instructions contains a Branch…

Handling Control Hazard

Time and Effort Lost with Wrong Prediction

- Stall until the branch outcome is known or perform the fetch again
- Predict the behavior of branches… better, with…
  - Static prediction by the compiler;
  - Dynamic prediction by the hardware:
    - Predict not taken approach
      Assume that the branch is not taken, i.e. the condition will not evaluate to be true
    - Predict taken approach
      Assume that the branch is taken, i.e. the condition will evaluate to be true
    - Delayed branching
      A more effective solution

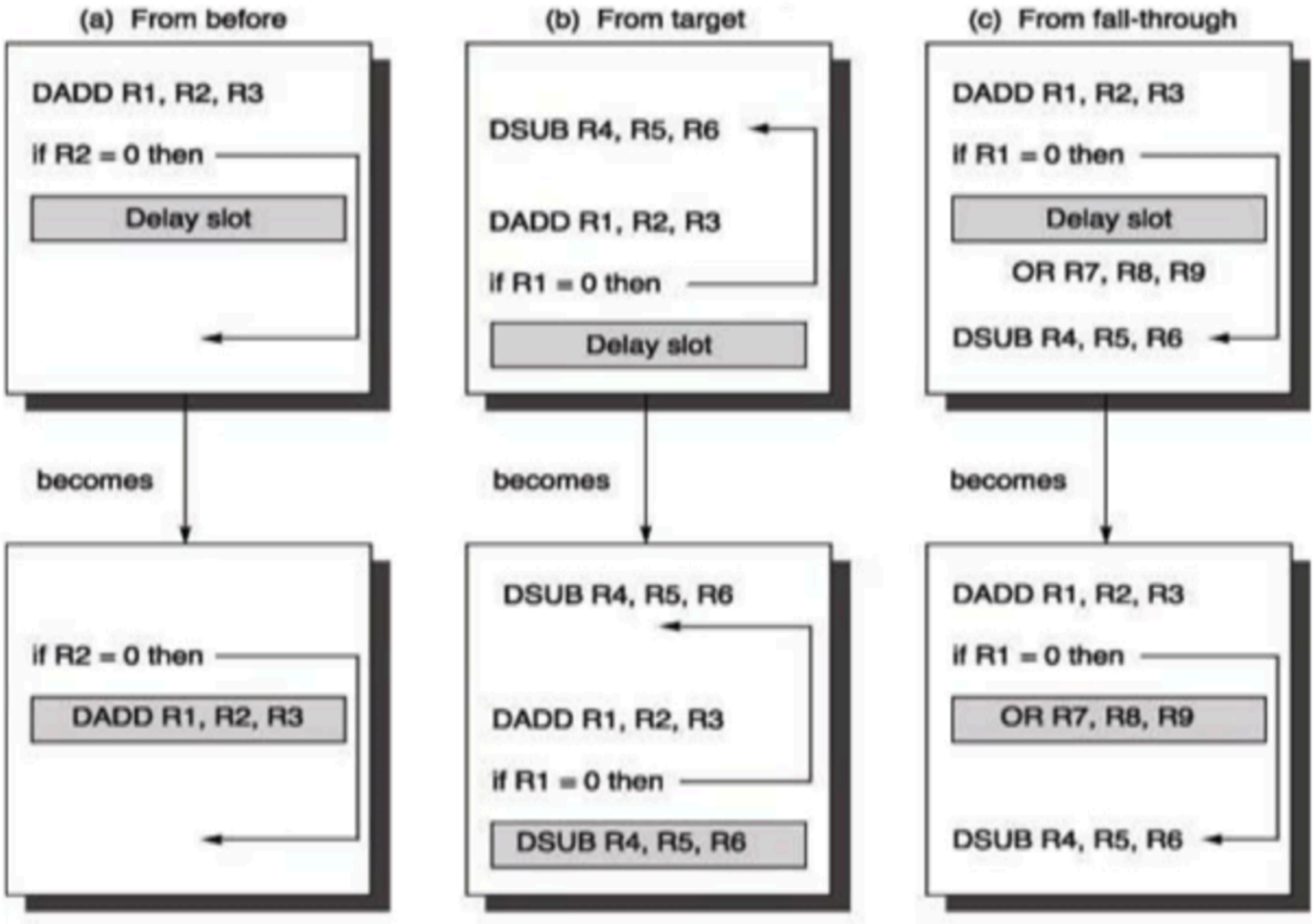
Branch Prediction

Better Branch Prediction: Delay Slots

- There are three different ways of introducing instructions in the delay slots:
  - Before branch instruction
  - From the target address: only valuable when branch taken
  - From fall through: only valuable when branch is not taken

- Plus, Cancelling branches allow more slots to be filled.

Ways to insert Delay Slots



Evaluating Hazards’ Effects

Pipeline Speed-Up

$$PipelineSpeedUp = \frac{PipelineDepth}{1 + PipelineStallCPI} = \frac{PipelineDepth}{1 + BranchFrequency * BranchPenalty}$$

Given an application where 20% of the instructions executed are conditional branches and 59% of those are taken. For the pipeline of our machine, what speedup will be achieved using a scheme where all branches are predicted as taken over a scheme with no branch prediction (i.e. branches will always incur a 1 cycle penalty)? Ignore all other stalls.

- CPI with no branch prediction = 0.8 x 1 + 0.2 x 2 = 1.2
- CPI with branch prediction = 0.8 x 1 + 0.2 x 0.59 x 2 + 0.2 x 0.41 x 1 = 1.118
- Speed up = 1.2 / 1.118 = 1.073

Pipeline Speed-Up

$$PipelineSpeedUp = \frac{PipelineDepth}{1 + PipelineStallCPI} = \frac{PipelineDepth}{1 + BranchFrequency * BranchPenalty}$$

Let us look at another example to calculate the speedup, given the following:

14% Conditional & Unconditional, 65% Taken; 52% Delay slots not usefully filled. The details for the various schemes are provided in the table below.

Scheduling Scheme	***Branch Penalty***	***CPI***	***Pipeline Speedup***	***Speedup vs stall***
Stall pipeline	3.00	1.42	3.52	1.00
Predict taken	1.00	1.14	4.39	1.25
Predict not taken	1.00	1.09	4.58	1.30
Delayed branch	0.52	1.07	4.66	1.32

Pipeline Speed-Up

$$PipelineSpeedUp = \frac{PipelineDepth}{1 + PipelineStallCPI} = \frac{PipelineDepth}{1 + BranchFrequency * BranchPenalty}$$

Let us look at an example to calculate the speedup, given the following:

14% Conditional & Unconditional, 65% Taken; 52% Delay slots not usefully filled. The details for the various schemes are provided in the table below.

Scheduling Scheme	***Branch Penalty***	***CPI***	***Pipeline Speedup***	***Speedup vs stall***
Stall pipeline	3.00	1.42	3.52	1.00
Predict taken	1.00	1.14	4.39	1.25
Predict not taken	1.00	1.09	4.58	1.30
Delayed branch	0.52	1.07	4.66	1.32

Stall: 1 + .14(branches) * 3(cycle stall)

Taken: 1 + .14(branches) * (.65(taken) * 1(delay to find address) + .35(not taken) * 1(penalty))

Not taken: 1 + .14 * (.65(taken) * 1 + [.35(not taken) * 0])

Delayed: 1 + .14 * (.52(not usefully filled) * 1)

Conclusion

Stall ‘em!

Pipeline Bubbling and Stalling…

Stupid, but effective.

Unfortunately, flushing the pipeline is still the most effective way of dealing with branch predictions, due to the cost and effort.

However, branch prediction is crucial at critical paths, or time sensitive processes.

It is also a hot topic in Computer Architecture for today, since it provides room for optimization, especially on mainframes and data centers.